



Instituto Tecnológico de San Juan del Río
INGENIERIA EN SISTEMAS COMPUTACIONALES

**RECONSTRUCCIÓN DEL CODIGO FUENTE (GHIDRA)
TOPICOS DE CIBERSEGURIDAD**

P R E S E N T A N:

Nombre del alumno

CRUZ GARCIA MIRNA PATRICIA

RESENDIZ MARTINEZ GALILEA

VALERIO CRISTINO CESAR

PROFESOR:

LEONARDO VALDES ARTEAGA

San Juan del Río, Querétaro a 14 de septiembre 2026.





INDICE

1.	INTRODUCCION	4
2.	METODOLOGIA	4
2.1	Extracción e importación en Ghidra	4
2.2	Rastreo e inspección del punto de entrada.....	5
2.3	Descomposición y análisis de la función principal	6
2.3.1	Obtención del código en C	6
2.3.1.1	Librerías estándar	6
2.3.1.2	Corrección de la firma de la función principal.....	6
2.3.1.3	Sustitución de tipos de datos no estándar:	7
2.3.1.4	Tratamiento de punteros y argumentos por línea de comandos:	7
2.3.1.5	Limpieza de formateo y cadenas de texto	7
2.4	Código en Ghidra y código en C.....	8
2.4.1	Uso de tcc	8
2.4.2	Comparación de ambos archivos	9
2.5	Mapeo de Traducción: Ghidra (C) a Python	9
2.6	Código en Python	11
2.6.1	Ejecución.....	11
3.	RESULTADOS	12

INDICE DE TABLAS

Tabla 1	Comparación tipos de datos en Ghidra y C.....	7
Tabla 2	Análisis de elementos del código de Ghidra y su traducción en Python	9
Tabla 3	Interpretación de resultados de salida y decodificación del ejecutable C vs. versión en Python.	12





INDICE DE ILUSTRACIONES

Ilustración 1 Descomprimido de la carpeta (password topicos)	4
Ilustración 2 Importación del archivo a trabajar (desconocido.exe).....	4
Ilustración 3 Desglose del contenido de la carpeta ejemplo2026	5
Ilustración 4 Ventana de opciones de análisis	5
Ilustración 5 INSPECCIÓN DE LA FUNCIÓN ENTRY Y FUN_00401000]	5
Ilustración 6 Código de la función FUN_00401000	6
Ilustración 7 Código original y código en C.....	8
Ilustración 8 Uso de tcc	8
Ilustración 9 Prueba con el código de python y el	11
Ilustración 10 Prueba con desconocido.exe	11



1. INTRODUCCION

La ingeniería inversa mediante desensambladores y descompiladores es un pilar fundamental en el análisis de software y la ciberseguridad. Permite examinar binarios cuando no se dispone del código fuente original con el fin de entender su comportamiento interno, auditar vulnerabilidades o validar algoritmos de cifrado y ofuscación.

En la presente práctica se analiza el ejecutable de 32 bits desconocido.exe utilizando la herramienta de análisis de código **Ghidra**. El objetivo es descompilar la función principal del programa, analizar la lógica de encriptación basada en operaciones matemáticas ASCII y reconstruir dicho comportamiento en el lenguaje **Python** (desmain.py), validando que la ejecución de la reconstrucción produzca resultados idénticos al binario original

2. METODOLOGIA

2.1 Extracción e importación en Ghidra

- Paso 1: Extracción del ejecutable: Se descomprimió el paquete protegido gh.zip mediante la clave correspondiente para obtener el archivo ejecutable *desconocido.exe*.

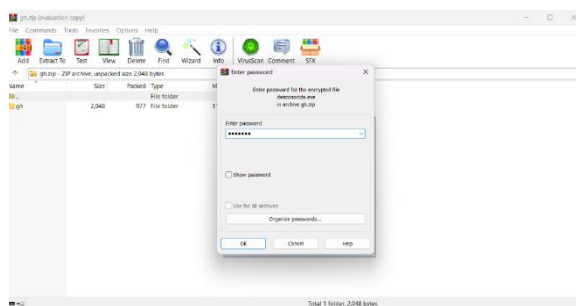


Ilustración 1 Descomprimido de la carpeta (password topicos)

- Paso 2. Creación del proyecto: Se creó un proyecto de análisis en Ghidra bajo el nombre de *ejemplo2026*
- Paso 3: Importación del binario: Al importar *desconocido.exe*, Ghidra identificó automáticamente las especificaciones técnicas del archivo (Portable Ejecutable PE, 32 bits, x86 Little-Endian)

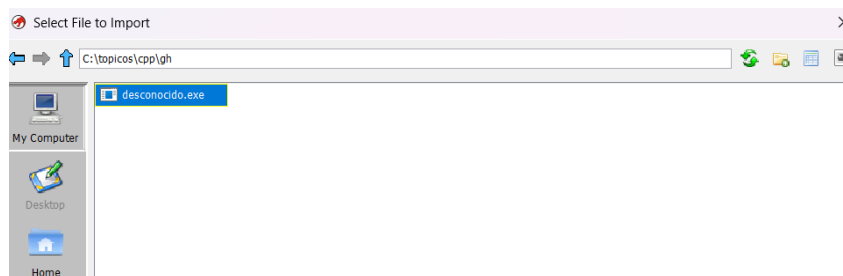


Ilustración 2 Importación del archivo a trabajar (desconocido.exe)

- Paso 4: Una vez importado se mostro la ventana ubicamos el nombre del archivo y al darle clic aparece una mini ventana como se muestra en la Ilustración 3

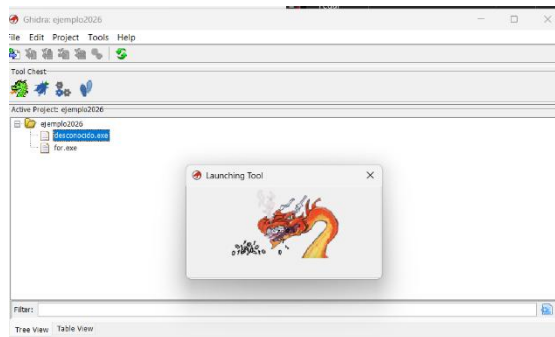


Ilustración 3 Desglose del contenido de la carpeta ejemplo2026

- Paso 5: Al abrirlo apareció el siguiente mensaje, se dio en yes

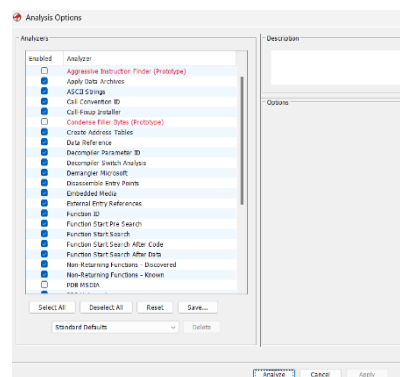
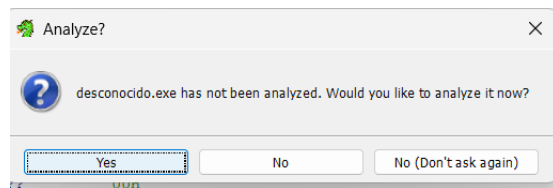


Ilustración 4 Ventana de opciones de análisis

2.2 Rastreo e inspección del punto de entrada

- Ubicación del Entry Point: En el panel de árbol de funciones, se navegó hacia el punto de entrada principal denominado entry.
- Análisis de la llamada principal: La descompilación de entry reveló la llamada a la función de usuario de alto nivel:

`_Code = FUN_00401000(*(int *)__argc_exref, *(int *)__argv_exref);`

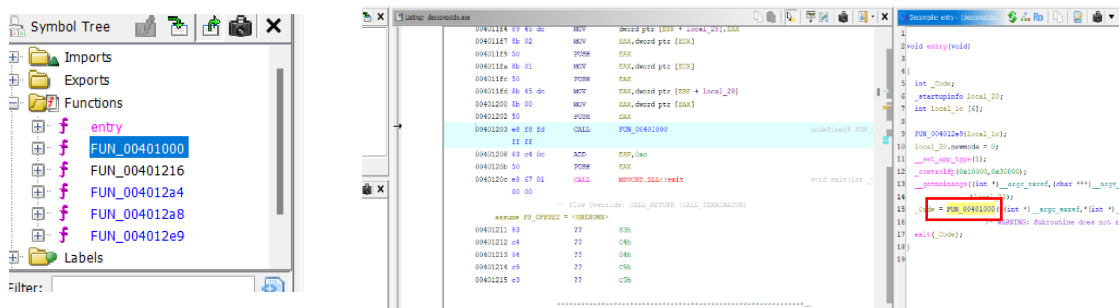


Ilustración 5 INSPECCIÓN DE LA FUNCIÓN ENTRY Y FUN_00401000]



2.3 Descomposición y análisis de la función principal

Al descompilar la función FUN_00401000, se observó un algoritmo que utiliza la función rand() para generar un flujo de datos ofuscado. La lógica se basa en comprar el valor ASCII de cada carácter del archivo de entrada contra un valor aleatorio de 8 bits (0-255).

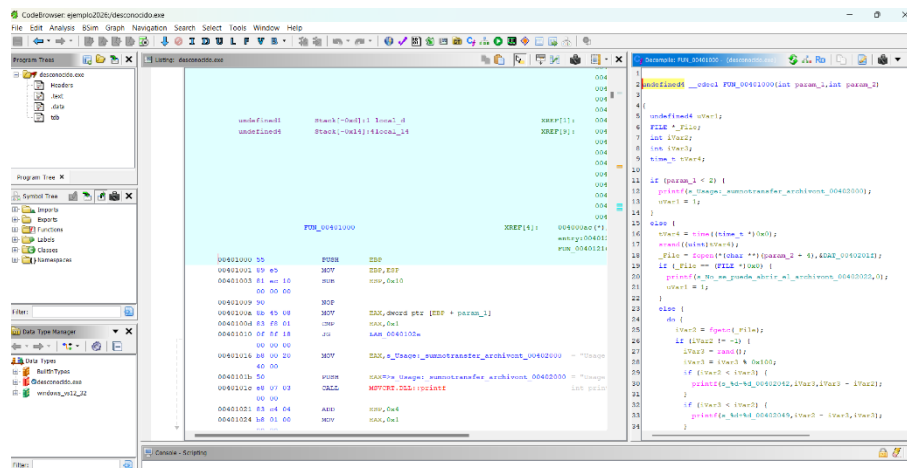


Ilustración 6 Código de la función FUN_00401000

2.3.1 Obtención del código en C

Para obtener el código fuente se copió y pegó en Visual Studio Code para incluir algunas librerías y hacer pequeñas modificaciones para adaptarlo a código C.

2.3.1.1 Librerías estándar

Como se puede observar Ghidra no incluye las cabeceras estándar de C por defecto en la vista del descompilador. Por lo que se agregaron:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
```

Esto para brindar soporte a funciones nativas como *printf*, *fopen*, *fgetc*, *fclose*, *rand* y *time*,

2.3.1.2 Corrección de la firma de la función principal

En Ghidra generalmente la firma utilizando etiquetas internas y convenciones de llamada explícitas de la arquitectura:

```
undefined4 __cdecl FUN_00401000(int param_1,int param_2)
```





Para adaptarlo a C estándar se remplazo por la firma estándar del punto de entrada en C.

```
int main(int param_1, char **param_2)
{
```

2.3.1.3 Sustitución de tipos de datos no estándar:

En Ghidra utiliza tipos de datos genéricos basados en el tamaño de palabra del procesador, como *undefined4* para variables de 32 bits.

En C estándar se sustituyeron por tipos de enteros nativos de C (*int*)

Tabla 1 Comparación tipos de datos en Ghidra y C

Ghidra	C
<pre>undefined4 uVar1; FILE *_File; int iVar2; int iVar3; time_t tVar4;</pre>	<pre>int uVar1; FILE *_File; int iVar2; int iVar3; time_t tVar4;</pre>

2.3.1.4 Tratamiento de punteros y argumentos por línea de comandos:

En el argumento de la línea de comandos mediante aritmética de punteros compleja se muestra:

```
_File = fopen(*(char **)(param_2 + 4), &DAT_0040201f);
if (_File == (FILE *)0x0) {
```

Para C se simplificó a la notación nativa de arreglos para acceder al primer argumento del programa:

```
_File = fopen(param_2[1], "rb");
if (_File == (FILE *)0x0) {
    printf("No se puede abrir el archivo\
```

2.3.1.5 Limpieza de formateo y cadenas de texto

Se remplazaron las referencias a etiquetas de memoria de Ghidra

```
printf(s_Usage: sumnotransfer_archivont_00402000);
uVar1 = 1;

lse {
    tVar4 = time((time_t *)0x0);
    srand((uint)tVar4);
    _File = fopen(*(char **)(param_2 + 4), &DAT_0040201f);
    if (_File == (FILE *)0x0) {
        printf(s_No_se_puede_abrir_el_archivont_00402022, 0);
        uVar1 = 1;
```





Por literales de cadena de estándar en C por ejemplo **"rb"** para el modo de lectura binaria y **"\n"** para los saltos de línea en la salida.

```
else {
    tVar4 = time((time_t *)0x0);
    srand((unsigned int)tVar4);
    _File = fopen(param_2[1], "rb");
    if (_File == (FILE *)0x0) {
        printf("No se puede abrir el archivo\n");
        uVar1 = 1;
    }
}
```

2.4 Código en Ghidra y código en C

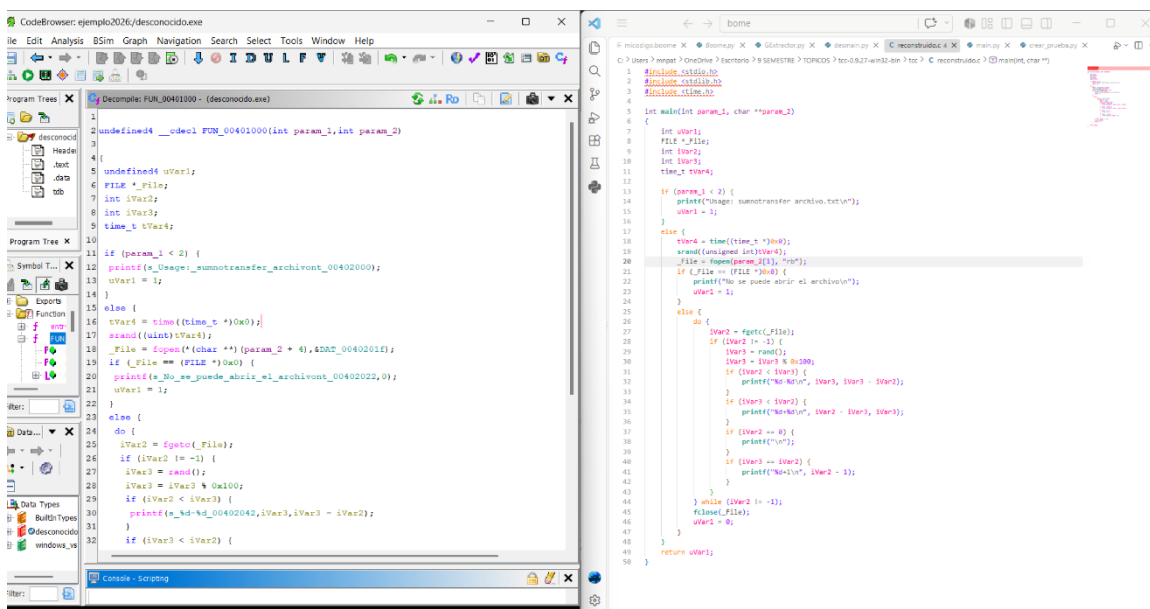


Ilustración 7 Código original y código en C

2.4.1 Uso de tcc

Con la ayuda de la herramienta tcc el nuevo archivo en C se pasó a un .exe como se muestra en la siguiente ilustración

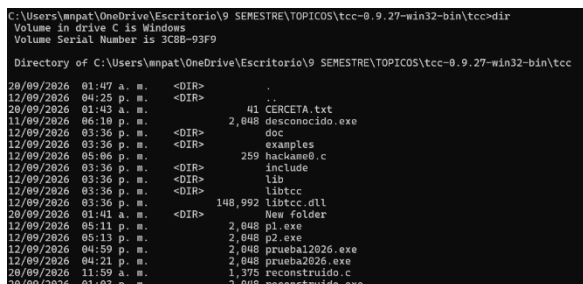


Ilustración 8 Uso de tcc





2.4.2 Comparación de ambos archivos

Instituto Tecnológico de San Juan del Río

```
C:\Users\mnpat\OneDrive\Escritorio\9 SEMESTRE\TOPICOS\tcc-0.9.27-win32-bin\tcc>desconocido.exe reconstruido.exe
199-122
287-117
19+125
17-17
1-1
168-165
96-96
1-1
116-116
1-1
113-113
1-1
249-245
87-87
1-1
13-13
1-1
109-109
1-1
11+244
35+220
97-97
1-1
8-8
1-1
24+168
```

2.5 Mapeo de Traducción: Ghidra (C) a Python

Tabla 2 Análisis de elementos del código de Ghidra y su traducción en Python

Componentes en Ghidra	Interpretación Técnica	Traducción en Python
undefined4 __cdecl FUN_00401000(int param_1, int param_2)	Función principal del programa. param_1 representa argc y param_2 representa argv.	def main(): y uso de sys.argv
if (param_1 < 2)	Comprueba si se proporcionó el nombre del archivo de entrada.	if len(sys.argv) < 2:
printf("Usage: ...")	Muestra el mensaje de uso cuando falta el argumento.	rint("Uso: python desmain.py <archivo.txt>")
FILE *_File; int iVar2; int iVar3;	Declara el archivo y las variables utilizadas para el byte leído y el número aleatorio.	archivo, iVar2, iVar3
fopen(..., "rb")	Abre el archivo en modo lectura binaria.	with open(nombre_archivo, "rb") as archivo:
if (_File == (FILE *)0x0)	Comprueba si el archivo no pudo abrirse.	except FileNotFoundError:
do { ... } while (iVar2 != -1)	Recorre el archivo hasta llegar al final.	while byte:
iVar2 = fgetc(_File)	Lee un byte del archivo.	byte = archivo.read(1)





iVar2 = fgetc(_File);	Convierte el byte leído a un valor entero.	iVar2 = byte[0]
iVar3 = rand() % 0x100	Genera un número pseudoaleatorio entre 0 y 255.	iVar3 = random.randint(0, 255)
if (iVar2 < iVar3)	Si el byte es menor que el aleatorio, calcula e imprime la diferencia.	if iVar2 < iVar3:
printf("%d-%d\n", iVar3, iVar3 - iVar2)	Imprime el aleatorio y la diferencia.	print(f"{iVar3}-{iVar3 - iVar2}")
if (iVar3 < iVar2)	Si el aleatorio es menor que el byte, calcula e imprime la diferencia y el aleatorio.	if iVar3 < iVar2:
printf("%d+%d\n", iVar2 - iVar3, iVar3)	Imprime la diferencia y el número aleatorio.	print(f"{iVar2 - iVar3}+{iVar3}")
if (iVar2 == 0)	Comprueba si el byte leído es 0x00.	elif iVar2 == 0:
printf("\n")	Imprime una línea vacía cuando el byte es cero.	pass
if (iVar3 == iVar2)	Comprueba si el número aleatorio es igual al byte.	elif iVar3 == iVar2:
printf("%d+1\n", iVar2 - 1)	Imprime el valor del byte menos uno seguido de +1.	print(f"{iVar2 - 1}+1")
fclose(_File)	Cierra el archivo después de terminar el procesamiento.	El bloque with open(...) cierra automáticamente el archivo.
return 0 / return 1	Indica éxito o error del programa.	main()





2.6 Código en Python

```

1  import sys
2  import random
3
4  def main():
5      if len(sys.argv) < 2:
6          print("Uso: python desmain.py <archivo.txt>")
7          return
8      nombre_archivo = sys.argv[1]
9      try:
10         with open(nombre_archivo, "rb") as archivo:
11             byte = archivo.read(1)
12
13             while byte:
14                 ivar2 = byte[0]
15                 ivar3 = random.randint(0, 255)
16
17                 if ivar2 < ivar3:
18                     print(f"{ivar3}-{ivar3 - ivar2}")
19                 elif ivar3 < ivar2:
20                     print(f"{ivar2 - ivar3}+{ivar3}")
21                 elif ivar3 == ivar2:
22                     print(f"{ivar2 - 1}+1")
23                 elif ivar2 == 0:
24                     pass
25
26                 byte = archivo.read(1)
27
28         except FileNotFoundError:
29             print(f"Error: No se pudo abrir el archivo '{nombre_archivo}'")
30
31 if __name__ == "__main__":
32     main()

```

2.6.1 Ejecución

```

C:\Users\mnpat\OneDrive\Escritorio\9 SEMESTRE\TOPICOS\cpp\gh>python desmain.py CERCETA.txt
169-100
201-120
19+66
31+42
75+5
248-169
69-37
154-87
55+14
173-91
116-49
234-167
23+46
107-23
129-64
191-126
135-70
85-20
110-64
152-139
239-229
127-54
27+83
221-105
140-39
41+62
48+66
9+88
50+60
22+94
204-143
222-107
194-136
148-135
216-206
182-20
5+64
212-129
113-44
84-6
22+46
143-78
248-158

```

Ilustración 9 Prueba con el código de python y el

archivo CERCETA.txt

```

C:\Users\mnpat\OneDrive\Escritorio\9 SEMESTRE\TOPICOS\cpp\gh>desconocido.exe CERCETA.txt
11+58
254-173
188-183
218-141
21+59
238-151
134-102
20+47
1+68
195-113
5+62
89-22
48+21
174-90
170-185
208-143
191-126
189-124
151-185
253-240
160-150
34+39
111-1
5+111
148-47
168-65
108+14
104-7
4+106
77+39
210-189
155-40
149-91
151-138
241-231
30+52
33+36
56+27
53+16
53+25
245-177
189-36

```

Ilustración 10 Prueba con desconocido.exe

y el archivo CERCETA.txt





3. RESULTADOS

VALIDACIÓN Y COMPARACIÓN DE RESULTADOS

Para verificar que la ingeniería inversa y la posterior reimplementación en Python fueron exitosas, se procesó el archivo de entrada CERCETA.txt tanto en el ejecutable original (desconocido.exe) como en el script en Python (desmain.py). Como se observa en la Tabla 3, aunque los valores aleatorios de las operaciones aritméticas difieren debido a la semilla aleatoria en cada ejecución (por ejemplo, 11+58 frente a 240-171), el valor numérico (código ASCII) resultante y el carácter equivalente recuperado son 100% idénticos en ambos casos (obteniendo el mensaje "EQUIPO CERCETA..."). Esto demuestra la total equivalencia funcional de ambos programas.

Tabla 3 Interpretación de resultados de salida y decodificación del ejecutable C vs. versión en Python.

desconocido.exe CERCETA.txt			python desmain.py CERCETA.txt		
OPERACION MATEMATICA	VALOR NUMERICO (CODIGO ASCII)	CARACTER EQUIVALENTE	OPERACION MATEMATICA	VALOR NUMERICO (CODIGO ASCII)	CARACTER EQUIVALENTE
11+58	69	E	240-171	69	E
254-173	81	Q	151-70	81	Q
188-103	85	U	77+8	85	U
214-141	73	I	235-162	73	I
21+59	80	P	142-62	80	P
230-151	79	O	204-125	79	O
134-102	32		79-47	32	
20+47	67	C	252-185	67	C
1+68	69	E	50+19	69	E
195-113	82	R	205-123	82	R
5+62	67	C	251-184	67	C
89-22	67	C	217-150	67	C
48+21	69	E	164-95	69	E
174-90	84	T	188-104	84	T
170-105	65	A	167-102	65	A
208-143	65	A	172-107	65	A
191-126	65	A	89-24	65	A
189-124	65	A	203-138	65	A
151-105	46	.	196-150	46	.
253-240	13		82-69	13	
160-150	10		232-222	10	
34+39	73	I	121-48	73	I
111-1	110	n	95+15	110	n

